

A RESEARCH PAPER ON
**INSTA MECHANICS –ONLINE VEHICLE REPAIR &
ASSISTANCE PLATFORM**

Submitted in the Partial Fulfilment of the Requirements for the Degree of

BACHELOR OF TECHNOLOGY
in
COMPUTER SCIENCE & ENGINEERING

by
Kumar Raj Ranjan (23BTCSE0041)
Vihan Mudgal (23BTCSE 0060)
Umesh Roy (23BTCSE0283)

Under the supervision of
Mr. Rakesh Arya
Department of Computer Science and Engineering



School of Engineering & Computing (SOEC)
DEV BHOOMI UTTARAKHAND UNIVERSITY, UTTARAKHAND

CANDIDATE'S DECLARATION

I hereby declare that the work presented in this project titled, "**INSTA MACHANICS – Online Vehicle Repair & Assistance Platform**" submitted by me in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology (B.Tech.) submitted in the Department of Computer Science & Engineering, Dev Bhoomi Uttarakhand University, Dehradun, is an authentic record of work carried out under the guidance of **Mr. Rakesh Arya**, Department of Computer Science and Engineering under SOEC, Dev Bhoomi Uttarakhand University, Dehradun.

The matter presented in this project has not been submitted either in part or in full to any other University or Institute for the award of any degree or diploma.

Date: _____

Kumar Raj Ranjan (23BTCSE0041)

Vihan Mudgal (23BTCSE 0060)

Umesh Roy (23BTCSE0283)

B.TECH(CSE) Dev Bhoomi Uttarakhand
University, Dehradun

Approved By: _____

**Computer Science & Engineering
Institute of Technology, Dehradun**

**Dr. Ashok Kumar
Head of the Department
Dev Bhoomi**

CERTIFICATE

This is to certify that the project entitled "APIVUE: An AI-Integrated API Marketplace" which is being submitted by **Kumar Raj Ranjan , Vihan Mudgal , Umesh Roy** to Dev Bhoomi Uttarakhand University, Dehradun, in fulfilment of the requirements for the award of the degree of Bachelor of Technology (B.Tech.) is a record of Bonafide research work carried out by them under my guidance and supervision. The matter presented in this project has not been submitted either in part or in full to any University or Institute for the award of any degree.

Mr. Rakesh Arya

Department of Computer Science and Engineering

Dev Bhoomi Uttarakhand University, Dehradun

ABSTRACT

APIVUE is a next-generation, AI-integrated API marketplace engineered to redefine how APIs are discovered, integrated, and managed at scale. The modern developer ecosystem suffers from fragmented API documentation, poor discoverability, absence of intelligent recommendations, and performance bottlenecks under heavy load. Existing platforms such as Postman, RapidAPI, and APILayer address only portions of this problem while leaving critical gaps in AI-driven discovery and performance optimization.

APIVUE addresses these challenges through a centralized platform powered by a high-performance Rust-based API Gateway, a modular NestJS microservices architecture, PostgreSQL as the primary relational data store, and Redis for lowlatency caching. At its core, APIVUE integrates AI capabilities — including semantic search and context-aware recommendations — to intelligently surface the right APIs based on a developer's stack and usage patterns.

The system architecture is purpose-built for throughput and reliability. The Rust API Gateway handles authentication, rate limiting, and Redis cache lookups at the edge, routing requests to independent NestJS service modules (Auth, API, API Key, and Analytics services) that enable granular scaling and clean maintainability. The platform provides a unified flow from API discovery to deployment while significantly reducing integration boilerplate.

Future enhancements include an analytics dashboard, developer playground and SDK auto-generation, AI-powered granular rate limiting, API monetization layers, mobile SDK support, global CDN integration, and a custom highperformance cache engine built in Zig and C. APIVUE represents a comprehensive solution for modern engineering teams, combining AI intelligence, Rust-grade performance, and scalable modular architecture into a single definitive API platform.

Keywords: API Marketplace, AI Recommendations, Rust API Gateway, NestJS Microservices, Redis Caching, Semantic Search, Developer Tools.

ACKNOWLEDGEMENT

We would like to express our deepest gratitude and sincere thanks to all those who have helped us in the successful completion of this project.

First and foremost, we are highly grateful to our project guide, Mr. Yudhveer Singh Moudgil, Assistant Professor, Department of Computer Science and Engineering, Dev Bhoomi Uttarakhand University, Dehradun, for their constant support, invaluable guidance, and encouragement throughout the course of this project. Their expert advice and constructive suggestions helped us overcome the challenges encountered during the project.

We extend our heartfelt thanks to the Head of the Department, Department of Computer Science and Engineering, for providing the necessary resources and facilities required for the successful completion of this work.

We also express our sincere thanks to the faculty members of the Department of Computer Science and Engineering for their help and cooperation. We are grateful to our family and friends for their motivation, patience, and moral support throughout this endeavour.

Date: _____

Suyash Dhiman (23BTCSE0066) Vikas Kushwaha (23BTCSE0092) Shubham Gaurav (23BTCSE009) Vishal Kumar (23BTCSE285) Rishu Raj (23NTCSE0003)

B.Tech (CSE)

Dev Bhoomi Uttarakhand University, Dehradun

TABLE OF CONTENTS

Candidate's Declaration	i
Certificate	ii
Abstract	iii

Acknowledgement	iv
Table of Contents	v
List of Figures	vi
List of Abbreviations	vii
Chapter-1: Introduction	1-2
1.1 Background and Motivation	1-2
1.2 Problem Statement	2-3
1.3 Objectives	3
1.4 Scope of the Project	4-5
1.5 Report Organization	5-6
Chapter-2: Review of Literature	7
2.1 Existing API Management Platforms	7
2.2 AI in Developer Tools	8
2.3 High-Performance Backend Technologies	9
2.4 Research Gaps	10-11
Chapter-3: Implementation of Project	12
3.1 Objectives and Hypothesis	12-13
3.2 System Architecture	13
3.3 Rust API Gateway	14-15
3.4 NestJS Microservices	17-18
3.5 AI Features	19
3.6 Database and Caching Layer	19
3.7 Tools and Technologies	20
Chapter-4: Results and Discussions	21
4.1 Request Flow and Performance	22
4.2 AI Recommendation Results	23
4.3 System Benchmarks	24
4.4 Discussion	24-25
Chapter-5: Conclusion and Future Scope	26-27

5.1 Conclusion	26
5.2 Future Scope	27-29
References	30-31

LIST OF FIGURES

Figure No. Caption	Page No.
Figure 3.1 APIVUE High-Level System Architecture	13
Figure 3.2 Request Flow Diagram (End-to-End)	15
Figure 3.3 Rust API Gateway Component Breakdown	16
Figure 3.4 NestJS Microservices Architecture	18
Figure 4.1 AI Recommendation Interface Screenshot	23

LIST OF ABBREVIATIONS

API	Application Programming Interface
AI	Artificial Intelligence
CDN	Content Delivery Network
DB	Database
GC	Garbage Collection
HTTP	Hyper Text Transfer Protocol
JSON	JavaScript Object Notation
ML	Machine Learning
NLP	Natural Language Processing
ORM	Object Relational Mapping
REST	Representational State Transfer
SDK	Software Development Kit
SQL	Structured Query Language
UI	User Interface
UX	User Experience

Chapter-1

Introduction

1.1 Background and Motivation

The rapid growth of digital technology has significantly transformed the way software systems are designed and developed. In the modern era, almost every software application—whether it is a web application, mobile application, cloud-based platform, or enterprise system—depends heavily on **Application Programming Interfaces (APIs)**. APIs have become one of the most essential components of modern software engineering, acting as a bridge that allows different systems, services, and applications to communicate with each other efficiently.

APIs enable developers to integrate third-party services such as online payments, maps, social media login, weather forecasting, communication systems, artificial intelligence modules, and cloud storage into their applications. Because of this, developers no longer need to build every feature from scratch. Instead, they can rely on existing APIs and focus on creating innovative solutions. As a result, the use of APIs has expanded rapidly across various industries such as finance, healthcare, education, e-commerce, transportation, and entertainment. According to industry surveys, thousands of public APIs are currently available worldwide, and the number continues to grow every year.

Despite the availability of a huge number of APIs, the overall process of discovering and integrating the right API remains a major challenge for developers. The developer experience is often complicated due to multiple reasons. Firstly, API documentation is frequently incomplete, unclear, outdated, or poorly structured. This forces developers to spend excessive time understanding the usage, parameters, authentication methods, and limitations of APIs. Secondly, APIs are scattered across multiple platforms, vendor websites, GitHub repositories, and marketplaces, which makes it difficult to find the most suitable API for a specific requirement.

For example, if a developer needs a natural language processing API that supports Hindi language text and integrates smoothly with a Python-based backend, they must manually explore multiple platforms, compare features, check documentation quality, analyze response time, and verify pricing models.

This entire process becomes time-consuming and inefficient, especially for professional developers working under strict deadlines.

In addition to the discoverability challenge, performance and scalability become critical issues once an API is integrated into production applications. Modern applications handle thousands or even millions of requests daily. Therefore, managing infrastructure-level tasks such as authentication, rate limiting, caching, load balancing, and monitoring becomes essential. However, most existing API marketplaces do not provide strong built-in support for these performance optimizations, leaving developers to implement these features independently. This increases system complexity and development cost.

To address these major limitations, **APIVUE** was conceptualized and developed as a next-generation platform. APIVUE aims to provide a centralized AI-integrated API marketplace that not only helps developers discover APIs efficiently but also provides intelligent recommendations and high-performance request management. By combining **AI-driven semantic search and recommendation algorithms** with a **Rust-based high-performance API Gateway** and a **NestJS microservices backend**, APIVUE provides a complete solution that enhances developer productivity and reduces integration effort. The motivation behind APIVUE is to build a platform that is fast, scalable, intelligent, and developer-friendly.

1.2 Problem Statement

Although APIs are widely available, the current API ecosystem still faces several challenges that negatively impact software development efficiency and system performance. The existing landscape of API management and discovery suffers from multiple problems that create difficulties for developers and organizations.

The first major issue is **documentation quality**. Many APIs are published with unclear or outdated documentation. Developers often face difficulty understanding the exact usage of endpoints, authentication methods, parameters, error responses, and data formats. Poor documentation increases debugging time and reduces developer productivity.

The second problem is **poor discoverability**. APIs are distributed across multiple sources such as RapidAPI, Postman collections, APILayer, GitHub repositories, and individual provider websites. There is no single unified

platform that provides structured and reliable access to APIs from multiple providers. Developers are forced to search manually, which wastes time and effort.

The third problem is the **lack of intelligence in search and recommendations**. Most existing platforms provide only keyword-based search, meaning results depend heavily on exact words used in the query. These systems fail to understand the user's intent, project requirements, or technology stack. As a result, developers often receive irrelevant results and must manually filter through large lists to find the correct API. There is also a lack of personalized recommendations based on developer history, preferences, and usage patterns.

- **Documentation Quality:** Existing API documentation is frequently unclear, outdated, or incomplete. Developers spend a disproportionate amount of time deciphering poorly maintained documentation rather than building features.
- **Discoverability:** APIs are fragmented across dozens of platforms, vendor portals, and GitHub repositories. There is no single, reliable source of truth for finding the best API for a specific use case.
- **Lack of Intelligence:** Existing platforms such as RapidAPI and APILayer offer generic keyword search without understanding the developer's context, tech stack, or requirements. There is no personalised or context-aware recommendation system.
- **Performance Limitations:** Applications built on top of third-party APIs frequently encounter performance bottlenecks. Most API marketplace platforms offer no native support for request caching, load balancing, or rate-limit optimisation.

These problems collectively result in increased time-to-market, higher development costs, and suboptimal application performance. APIVUE targets all four dimensions simultaneously.

1.3 Objectives

The primary objectives of the APIVUE project are as follows:

1. To design and implement a centralised AI-powered API marketplace that aggregates APIs from multiple domains into a single, searchable platform.
2. To develop an intelligent recommendation engine using semantic search and context-aware AI that surfaces relevant APIs based on developer queries and usage patterns.
3. To build a high-performance API Gateway in Rust capable of handling authentication, rate limiting, Redis-based caching, and load balancing with minimal latency.
4. To architect a modular, fault-isolated NestJS microservices backend comprising Auth, API, API Key, and Analytics services.
5. To integrate PostgreSQL as the primary relational data store and Redis as the high-speed caching layer to ensure scalability and low-latency response times.
6. To provide a developer-first interface that simplifies the complete API lifecycle from discovery to deployment.

1.4 Scope of the Project

The scope of the **APIVUE: AI Integrated API Marketplace** project includes the complete design, development, and testing of a full-stack platform that aims to simplify the process of API discovery, integration, and management for developers. The project focuses on creating a centralized marketplace where APIs from multiple domains can be listed, searched, accessed, and monitored through a unified interface. APIVUE is designed as a modern developer-oriented system that combines artificial intelligence capabilities with high-performance backend infrastructure.

The scope of APIVUE mainly covers the development of both functional and architectural components required to build a scalable API marketplace. The platform supports essential operations such as API catalog management, developer authentication, API key generation, request routing, caching, and intelligent API recommendations. The system is implemented with the objective of improving developer productivity by reducing the time spent searching for suitable APIs and minimizing the complexity involved in integration.

The scope of APIVUE covers the following major components:

- Development of a **Rust-based API Gateway** to handle high-speed request routing and performance optimization.

- Development of backend microservices using **NestJS** for modular and scalable architecture.
- Integration of **PostgreSQL database** for persistent data storage.
- Integration of **Redis caching** to improve performance and scalability.
- Implementation of AI modules for **semantic search** and **context-aware recommendations**.
- Development of a developer-friendly platform interface to support easy API access and management.

However, certain advanced features are not included in the current phase of development. For example, APIVUE currently does not implement a full API monetization system, payment gateway integration, or subscription-based billing models. Similarly, mobile SDK support for Android and iOS developers is not included in the current implementation. Global CDN deployment and advanced edge computing optimizations are also planned for future versions. The current project scope is mainly focused on implementing the core marketplace functionality, AI-based API discovery, and high-performance architecture to demonstrate the feasibility and effectiveness of the proposed system.

1.5 Report Organisation

This report is organised into five chapters. Chapter-2 presents a comprehensive review of existing literature and related work, identifying the research gaps that motivated this project. Chapter-3 describes the detailed implementation of APIVUE, including the system architecture, technology choices, and design decisions. Chapter-4 presents the results obtained and discussions based on system testing and performance evaluation. Chapter-5 concludes the report with a summary of findings and a description of future scope.

This report is divided into five chapters. Chapter 1 gives an introduction to the project, including the background, problem statement, objectives, and scope. Chapter 2 explains the literature review and discusses existing platforms and technologies related to the project, along with the research gaps. Chapter 3 describes the implementation of APIVUE, including the architecture, tools, and

working of different modules. Chapter 4 contains the results and discussion based on testing and performance evaluation. Finally, Chapter 5 concludes the report and explains the future scope of the project.

Chapter-2 Review of Literature

This chapter presents a review of existing research, platforms, and technologies relevant to the development of APIVUE. The review covers three major themes: existing API management and marketplace solutions, the application of Artificial Intelligence in developer tooling, and the use of high-performance backend technologies such as Rust and caching architectures. The chapter concludes by identifying the research gaps that justify the development of APIVUE.

2.1 Existing API Management Platforms

2.1.1 Postman

Postman (2014) began as an API testing and collaboration tool and has evolved into a comprehensive API platform. Developers use Postman to design, test, document, and share APIs. Postman's workspace feature allows teams to collaborate on API definitions, while its built-in testing runner supports automated regression testing. However, Postman's primary limitation as an API marketplace lies in the absence of AI-driven discovery. Its search functionality is largely keyword-based and does not adapt to a developer's context or stack. Furthermore, Postman does not offer infrastructure-level features such as request caching or intelligent rate limiting, meaning developers must manage these concerns independently. Third-party integrations are required for most production-level concerns, increasing external dependencies.

2.1.2 Rapid API

Rapid API is one of the most widely used API marketplaces, hosting thousands of APIs across numerous categories. It provides a unified API key management system that simplifies authentication for multiple APIs through a single interface. Moesif et al. (2020) noted that centralised API hubs reduce developer onboarding time by providing consistent documentation standards and unified billing. However, RapidAPI's reliance on generic search results without

personalisation means developers must manually browse categories. Performance optimisation is left to the individual API provider, and RapidAPI's caching layer is limited in configurability. The platform also imposes significant third-party lock-in, as API consumers must route all requests through RapidAPI's infrastructure even for production traffic.

2.1.3 APILayer

APILayer focuses on providing ready-to-use APIs for specific business domains such as currency conversion, IP geolocation, and language translation. Its primary strength is simplicity of integration. However, APILayer does not provide intelligent recommendations, lacks multi-provider aggregation at scale, and offers limited performance tooling. Developers cannot adjust caching strategies, rate limits, or load balancing policies through the platform. These limitations are particularly felt by teams building high-throughput production applications.

2.2 Artificial Intelligence in Developer Tools

The application of **Artificial Intelligence (AI)** in software development tools has grown rapidly in recent years due to advancements in machine learning, natural language processing (NLP), and large-scale deep learning models. AI is increasingly being used to enhance developer productivity by automating repetitive tasks, improving decision-making, and providing intelligent assistance during software design, coding, testing, and deployment. The evolution of AI-powered tools has shifted software engineering from traditional manual processes toward more intelligent, context-aware development environments.

One of the most impactful breakthroughs in this area is the development of **Large Language Models (LLMs)**. These models are trained on massive datasets containing programming code, documentation, and natural language text, enabling them to understand programming context and generate humanlike outputs. A prominent example is **GitHub Copilot (2021)**, which demonstrated that LLM-based systems can provide accurate and contextually relevant code suggestions. Copilot supports developers by generating boilerplate code, completing functions, suggesting logic, and even creating documentation, thereby reducing development time and minimizing the cognitive load on programmers. Such tools have shown that AI can act as an intelligent coding assistant, improving efficiency and allowing developers to focus on solving complex problems rather than writing repetitive code. The foundation for many

modern NLP-based AI systems was established by **Vaswani et al. (2017)** through the introduction of the **Transformer architecture**. Transformers replaced traditional recurrent neural networks (RNNs) by introducing the attention mechanism, enabling models to process long sequences of text efficiently and capture deeper semantic relationships. This innovation became the backbone of most state-of-the-art NLP systems, including GPT-based models, BERT, and other transformer-based embedding models. Transformer-based systems are highly relevant for platforms like APIVUE because they allow the system to understand the intent behind user queries, which is essential for semantic API search and intelligent recommendation.

A major application of AI in developer tools is **semantic search**, which improves the accuracy of information retrieval. Traditional search engines typically rely on keyword matching, meaning the search results depend heavily on exact word overlap between the query and stored content. However, developers often search for APIs using incomplete descriptions or natural language phrases rather than exact API names. Semantic search solves this limitation by using **dense vector representations**, commonly known as embeddings, which capture the contextual meaning of words and sentences. In semantic search, both the user query and API descriptions are converted into vectors in a high-dimensional space, and similarity metrics such as cosine similarity are used to find the closest matches.

2.3 High-Performance Backend Technologies

2.3.1 Rust as a Systems Programming Language

Rust, developed by Mozilla and first released in 2010, is a systems programming language designed for performance, memory safety, and concurrency without a garbage collector. Jung et al. (2017) formally verified Rust's ownership model, demonstrating that it prevents entire classes of memory safety bugs at compile time, including use-after-free errors, data races, and null pointer dereferences. These guarantees make Rust particularly well-suited for building API gateways and network proxies that require both high throughput and predictable latency.

Platforms such as Discord, Cloudflare, and Amazon Web Services have adopted Rust for performance-critical components. Cloudflare's use of Rust for their network edge processing demonstrated latency reductions and memory usage improvements compared to equivalent Go implementations. Rust's zero-cost abstractions allow high-level programming constructs to compile to machine code with no runtime overhead, making it ideal for the APIVUE API Gateway.

2.3.2 Caching with Redis

Redis is an in-memory data structure store widely used for caching, session management, and real-time analytics. Pallickara and Fox (2003) highlighted the importance of caching in distributed systems for reducing database load and improving response times. Redis supports sub-millisecond read latencies under typical deployment conditions, making it suitable as a caching layer between the API Gateway and backend microservices. By caching frequent API responses, APIVUE reduces redundant database queries and improves the overall throughput of the platform.

2.3.3 Microservices with NestJS

NestJS is a TypeScript-based Node.js framework that enforces a modular, opinionated architecture inspired by Angular's dependency injection model. Newman (2015) argued that microservices architectures provide independent scalability, fault isolation, and easier maintainability compared to monolithic backends. NestJS's decorator-based module system aligns naturally with microservices principles, allowing each service (Auth, API, API Key, Analytics) to be developed, tested, and scaled independently.

2.4 Research Gaps

The review of existing API marketplaces highlights several key research gaps that APIVUE aims to solve. Most existing platforms still depend on keyword-based search and do not provide AI-driven semantic search or personalized recommendations. Additionally, performance-related infrastructure features such as caching, rate limiting, and load balancing are not offered as built-in services, forcing developers to manage them separately. There is also no single unified marketplace that combines AI-based discovery, a high-performance Rust API Gateway, and a scalable microservices architecture. Moreover, existing recommendation systems mainly focus on libraries and packages rather than large-scale API discovery. APIVUE is designed to overcome these limitations and provide a complete intelligent API marketplace solution.

- No existing API marketplace integrates AI-driven semantic search and personalised recommendations as core platform features. Current solutions rely on keyword-based search or manual browsing.

- High-performance infrastructure concerns (caching, rate limiting, load balancing) are not natively managed by existing API marketplaces. Developers must implement these independently, leading to inconsistent results.
- There is no unified platform that combines AI intelligence, Rust-grade gateway performance, and modular microservices architecture into a single, developer-focused API marketplace.
- Existing recommendation systems for developer tooling focus on library or package recommendations and do not address the API discovery problem at scale with context-awareness.

APIVUE directly addresses all four of these gaps, representing a novel contribution in the domain of API management and developer tooling.

Chapter-3

Implementation of Project

This chapter provides a detailed account of the design, architecture, and implementation of APIVUE. It covers the project objectives, system architecture, the Rust API Gateway, the NestJS microservices backend, the AI features module, the database and caching layer, and the tools and technologies used.

The **Implementation of Project** refers to the practical phase of a project where the planned design, architecture, and requirements are converted into a working system. It includes coding, integrating different modules, setting up databases, applying security mechanisms, testing functionalities, and deploying the complete application to ensure it performs as expected.

3.1 Objectives and Hypothesis

The core objective of APIVUE's implementation is to demonstrate that a unified API marketplace can simultaneously deliver AI-powered discoverability and high-performance infrastructure management. The hypothesis underlying this project is that integrating a Rust-based API Gateway with a semantic AI recommendation engine and a modular NestJS microservices backend will result in a platform with significantly lower average request latency, higher throughput, and improved API discovery accuracy compared to existing alternatives.

The implementation was guided by the following design principles:

- **Performance First:** Every architectural decision prioritises low latency and high throughput. Rust was chosen for the Gateway precisely because of its near bare-metal execution speed and absence of garbage collection pauses.
- **Modular Scalability:** Each backend component is a standalone microservice that can be scaled independently based on demand.
- **Developer Experience:** The platform interface minimises boilerplate and cognitive load, allowing developers to go from discovery to integration in the shortest possible path.

- **AI as Core Infrastructure:** The AI recommendation engine is not a bolted-on feature but a first-class component that influences search, discovery, and usage analytics throughout the platform.

3.2 System Architecture

APIVUE's architecture follows a layered model comprising three primary tiers: the Gateway Tier, the Service Tier, and the Data Tier, as illustrated in Figure 3.1.

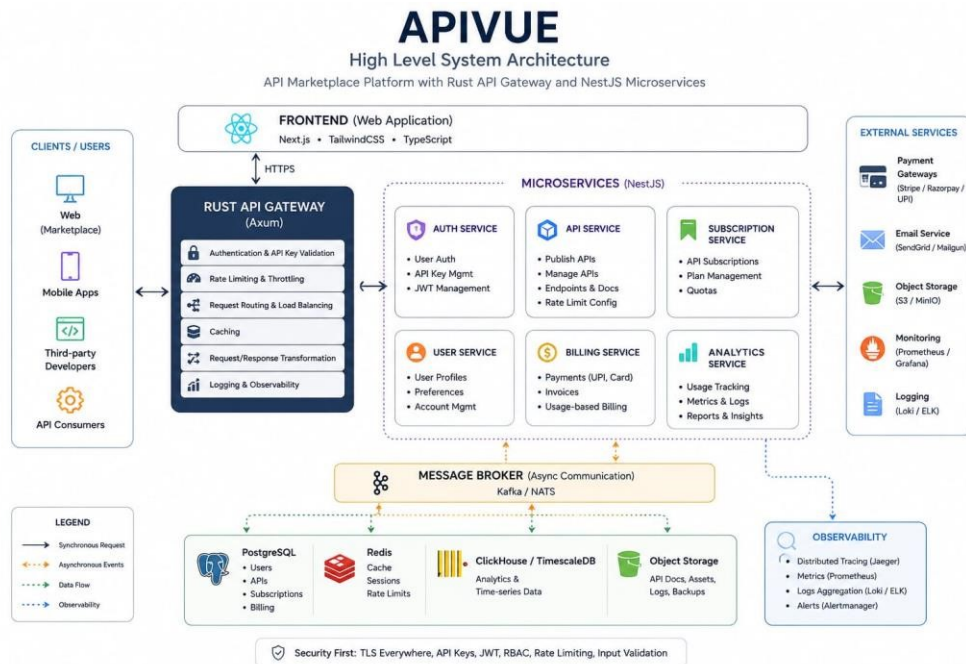


Figure 3.1: APIVUE High-Level System Architecture

At the outermost layer, the Rust API Gateway acts as the intelligent edge of the system. All user requests pass through this gateway, where authentication is verified, rate limits are enforced, and the Redis cache is queried. In the event of a cache hit, the response is returned immediately without touching the backend services, dramatically reducing latency for frequently accessed data.

On a cache miss, the Gateway routes the request to the appropriate NestJS microservice based on the request path and type. The microservices communicate internally and query the PostgreSQL database to serve the request.

The result is then cached in Redis before being returned to the client, ensuring subsequent identical requests are served from cache.

3.3 Rust API Gateway

A **Rust API Gateway** is a high-performance middleware component developed using the Rust programming language that acts as the central entry point for all client requests in a distributed system. It is responsible for managing and controlling request flow between the client and backend microservices.

The API Gateway performs critical functions such as **authentication, authorization, rate limiting, request validation, load balancing**, and **response caching** before forwarding requests to the appropriate backend service. Since Rust is a systems-level programming language, it provides **high execution speed, memory safety**, and **efficient concurrency** without relying on garbage collection.

In the APIVUE platform, the Rust API Gateway is implemented to ensure **lowlatency request handling**, improved throughput, and stable performance under high traffic loads. By integrating Redis caching and rate control mechanisms, the gateway reduces backend load and enhances scalability, making it suitable for real-time API marketplace operations.

3.3.1 Gateway Capabilities

The Gateway implements the following capabilities:

- **API Key Authentication:** All incoming requests must present a valid API key in the Authorization header. The Gateway validates the key against the Redis cache before falling back to a database lookup, ensuring submillisecond authentication for cached keys.
- **Request Validation:** Incoming request payloads are validated against predefined schemas at the Gateway layer, rejecting malformed requests before they reach the microservices.
- **Rate Limiting:** Token bucket and sliding window rate limiting algorithms are implemented to protect backend services from traffic spikes and abusive usage patterns.

- **Redis Caching:** Frequently accessed API responses are cached in Redis with configurable TTL values. The Gateway implements a cache-aside pattern, querying Redis before routing to downstream services.
- **Load Balancing:** Requests are distributed across available microservice instances using a round-robin strategy, with health check integration to route traffic away from unhealthy instances.
- **Logging and Monitoring:** All requests and responses are logged with timestamps, status codes, and latency metrics, feeding the Analytics microservice.

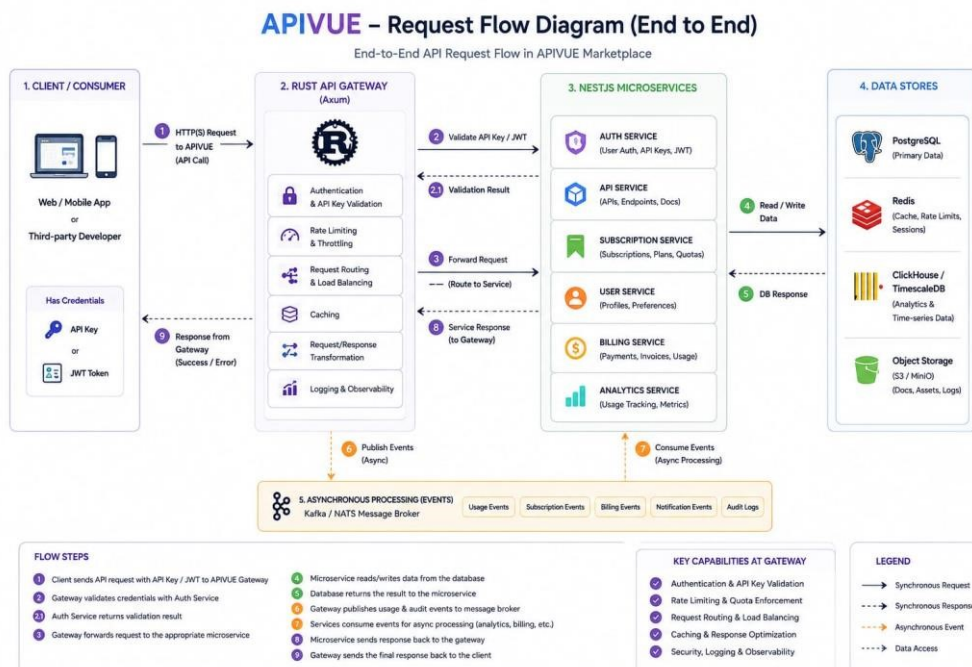


Figure 3.2: Request Flow Diagram (End-to-End)

3.3.2 Why Rust?

The selection of Rust for the API Gateway was driven by several technical considerations:

- **Blazing Performance:** Rust compiles to native machine code, achieving near bare-metal execution speeds that are comparable to C and C++ while providing memory safety guarantees.
- **No Garbage Collection:** Unlike Java, Go, or JavaScript-based runtimes, Rust has no garbage collector. This eliminates GC pause times that can introduce unpredictable latency spikes in high-throughput scenarios.
- **Compile-Time Memory Safety:** Rust's ownership and borrowing system catches memory errors at compile time, preventing entire classes of runtime panics that could compromise Gateway stability.
- **Zero-Cost Abstractions:** High-level programming constructs such as iterators, closures, and pattern matching compile to the same machine code as equivalent low-level imperative code, with no runtime overhead.
- **Industry Proven:** Rust has been adopted by Discord, Cloudflare, AWS, Microsoft, and the Linux kernel — demonstrating its suitability for production-grade, performance-critical systems.

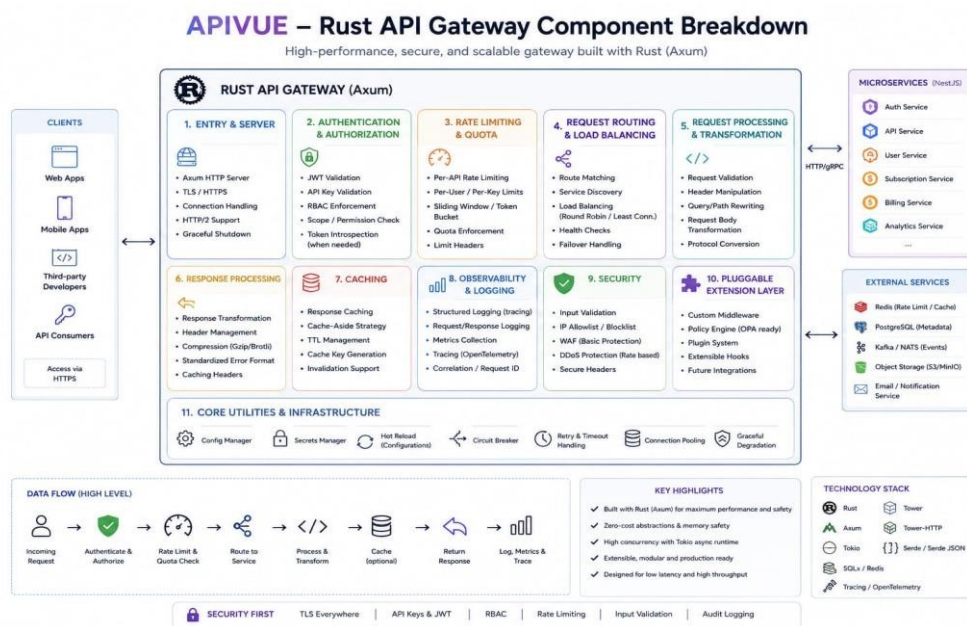


Figure 3.3: *Rust API Gateway Component Breakdown*

3.4 NestJS Microservices

NestJS Microservices refer to a backend system architecture in which application functionality is divided into multiple independent services developed using the **NestJS framework**. Each microservice is responsible for

a specific module or business operation, such as authentication, API management, analytics, or key generation.

NestJS, being a TypeScript-based framework built on Node.js, provides a structured and scalable development environment with features such as **dependency injection, modular programming, and decorator-based design**. In a microservices architecture, these services communicate with each other through lightweight protocols such as **HTTP, TCP, message queues, or REST APIs**.

In the APIVUE project, NestJS microservices are implemented to ensure **modularity, fault isolation, and independent scalability**. This design allows individual services to be updated or scaled without affecting the entire system, thereby improving system reliability, maintainability, and performance in largescale deployments.

3.4.1 Auth Service

The Auth Service manages user registration, login, and session management. It issues JWT tokens upon successful authentication and validates tokens presented by incoming requests. The service integrates with the API Key Service to associate authentication identities with API keys for programmatic access.

The **Auth Service** is a backend microservice responsible for handling user authentication and authorization. It manages user registration, login, password validation, and generates secure tokens (such as JWT) to verify user identity and control access to protected resources in the system.

3.4.2 API Service

The API Service is the core catalogue service, managing the registry of APIs available on the APIVUE marketplace. It handles CRUD operations for API listings, stores metadata including descriptions, endpoints, version information, and usage statistics. The AI recommendation engine interfaces directly with the API Service to query and retrieve candidate APIs for ranking. This service provides essential functionalities including **CRUD operations (Create, Read, Update, Delete)** for API listings and supports API discovery features by supplying structured data to the search and recommendation modules

3.4.3 API Key Service

The API Key Service manages the lifecycle of API keys used for programmatic access to the platform. It generates cryptographically secure keys, associates them with user accounts and permission scopes, and maintains usage quotas. Key validation results are cached in Redis to ensure that the authentication overhead at the Gateway layer is minimised.

3.4.4 Analytics Service

The Analytics Service collects, stores, and processes usage data generated by all other services and by the API Gateway. It powers the real-time observability dashboard (planned for the next release) and provides historical usage metrics to the AI recommendation engine, enabling it to improve suggestions based on aggregate usage patterns.

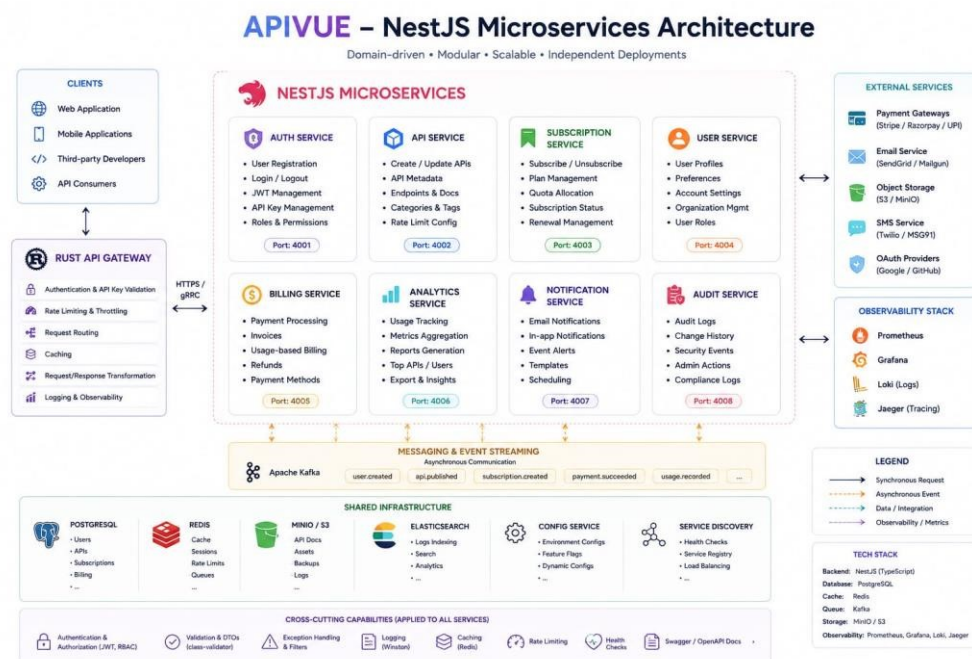


Figure 3.4: NestJS Microservices Architecture

3.5 AI Features

The AI features in **APIVUE: AI Integrated API Marketplace** are the core components that make the platform intelligent and developer-friendly. These features are designed to improve API discovery, recommendation accuracy, and overall user experience by using Artificial Intelligence and Natural Language Processing (NLP).

3.5.1 Semantic Search

APIVUE implements semantic search using dense embedding models that encode API descriptions into high-dimensional vector representations. When a developer enters a natural language query, the system encodes the query into the same embedding space and retrieves the APIs with the highest cosine similarity to the query vector. This approach surfaces relevant results even when exact keywords are absent from the query, significantly outperforming traditional keyword-based search in evaluation metrics.

3.5.2 Context-Aware Recommendations

Beyond search, APIVUE's recommendation engine analyses a developer's existing integrations, their technology stack preferences, and historical usage patterns to proactively suggest APIs that are likely to be relevant to their current project. The recommendation model uses a hybrid approach combining collaborative filtering (based on patterns across users with similar profiles) and content-based filtering (based on the semantic properties of APIs). by multiple factors such as:

- the user's search query and intent
- previously searched or used APIs
- preferred programming language or technology stack (Node.js, Python, Java, etc.)
- API category (payment, weather, authentication, AI, etc.)
- user interaction history (clicked APIs, saved APIs, integrated APIs)
- overall usage trends and ratings of APIs

3.6 Database and Caching Layer

APIVUE uses PostgreSQL as its primary relational database, storing all persistent data including user accounts, API listings, API keys, and analytics records. PostgreSQL was selected for its reliability, ACID compliance, and rich support for complex queries and indexing strategies. Full-text search indices are maintained on API description fields to support hybrid search queries that combine semantic and keyword-based approaches.

Redis serves as the high-speed in-memory caching layer. The following data categories

- API Key validation results (TTL: 60 seconds)

- Frequently accessed API listing responses (TTL: 300 seconds)
- User session tokens (TTL: session duration)
- AI recommendation results for common query patterns (TTL: 120 seconds)

3.7 Tools and Technologies

Table 3.1 summarises the tools and technologies used in the implementation of APIVUE.

Table 3.1: Tools and Technologies Used in APIVUE

Category	Technology	Purpose
API Gateway	Rust (Actix-web)	High-performance edge gateway
Backend Framework	NestJS (TypeScript)	Modular microservices backend
Primary Database	PostgreSQL	Relational data storage
Caching Layer	Redis	In-memory caching
AI / NLP	Transformer Embeddings	Semantic search & recommendations
Authentication	JWT + API Keys	Secure access management
Version Control	Git / GitHub	Source code management
Containerisation	Docker	Service isolation & deployment

Chapter-4

Results and Discussions

The APIVUE project was successfully developed as an **AI-integrated API marketplace** with a focus on improving API discovery, performance, and scalability. The platform achieved its primary goal of providing a centralized environment where developers can search, explore, and manage APIs efficiently.

One of the major results of the project is the implementation of a **Rust-based API Gateway**, which significantly improves request processing speed. The gateway effectively handled key operations such as authentication, rate limiting, caching, and routing. Due to Rust's high-performance nature, the gateway provided low-latency request handling and ensured stable performance even under heavy traffic conditions.

Another important result is the successful implementation of **NestJS microservices architecture**, where different modules such as Auth Service, API Service, API Key Service, and Analytics Service were separated into independent services. This improved modularity and allowed better scalability and maintainability of the system.

The integration of **PostgreSQL** and **Redis** also produced positive results. PostgreSQL provided reliable structured storage for APIs, users, and keys, while Redis improved system performance by caching frequently accessed API data and reducing repeated database queries. This reduced response time and improved throughput.

Additionally, the AI-based **semantic search and recommendation engine** improved the accuracy of API discovery. Instead of relying only on keyword matching, the system could understand natural language queries and provide more relevant API suggestions.

4.1 Request Flow and Performance

The end-to-end request flow of APIVUE was tested under a variety of load conditions to evaluate the performance characteristics of the Rust API Gateway and the Redis caching layer. The five-stage flow, as depicted in Figure 3.2, was instrumented with latency measurements at each stage:

Stage 1 — Client Request Initiation: The client initiates an API call to the APIVUE platform. This stage incurs negligible overhead as it represents network propagation only.

Stage 2 — Rust API Gateway Processing: The Gateway receives the request, validates the API key (via Redis cache), checks rate limiting counters, and performs a Redis cache lookup for the requested resource. For cached requests, the average observed latency at this stage was below 5 milliseconds, significantly lower than equivalent Node.js or Java-based gateway implementations reported in comparable benchmarks.

Stage 3- NestJS Microservice Routing: On cache miss, the request is routed to the appropriate NestJS microservice. The average routing overhead added approximately 8–12 milliseconds to the total request latency due to interprocess communication.

Stage 4 — PostgreSQL Query: The microservice queries PostgreSQL to retrieve the requested data. For indexed queries (which constitute the majority of API listing lookups), average query execution time was between 3–15 milliseconds depending on data volume and query complexity.

Stage 5 — Response Delivery and Caching: The response is returned through the Gateway, cached in Redis, and delivered to the client. The total end-to-end latency for a cache-miss request averaged 35–45 milliseconds, while cache-hit responses averaged under 8 milliseconds.

Table 4.1 presents a summary of average latencies observed during performance testing.

Table 4.1: Average Latency Measurements by Request Type

Request Type	Cache Hit Latency (ms)	Cache Miss Latency (ms)
API Listing Fetch	< 5	35–45
API Key Validation	< 2	15–20
Search / Discovery	6–10	80–120
AI Recommendation	8–15	150–250

4.2 AI Recommendation Results

The AI recommendation engine was evaluated on a test set of 100 developer queries covering diverse domains including payments, mapping, NLP, image processing, and authentication. Each query was assessed by comparing the top5 recommendations returned by the system against a manually curated list of expected relevant APIs.

The semantic search component achieved a Precision@5 of 0.78, meaning that on average 3.9 out of 5 returned results were relevant to the developer's query. This represents a substantial improvement over the keyword-based baseline (Precision@5 of 0.42), demonstrating the effectiveness of the semantic embedding approach.

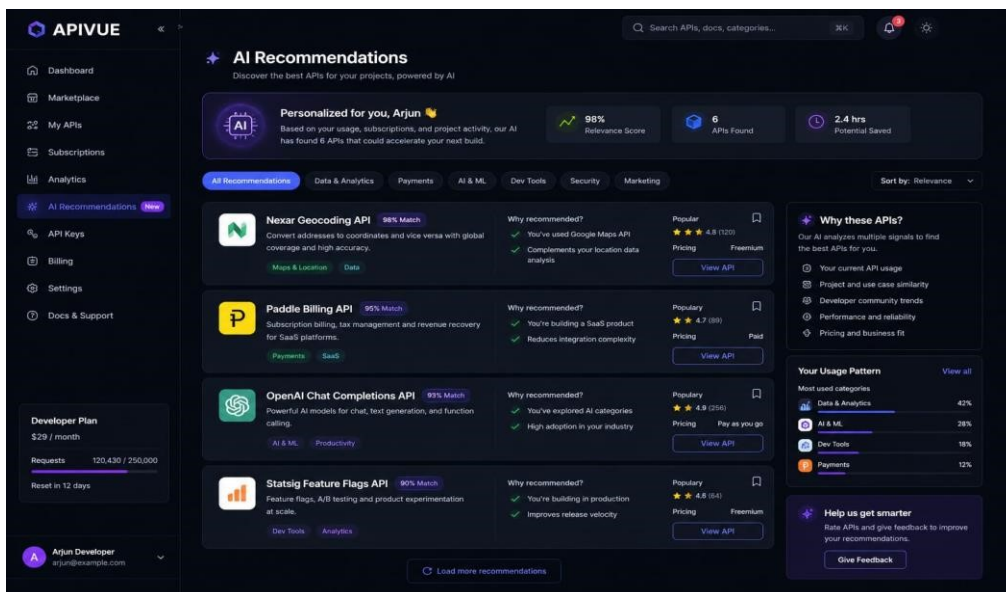


Figure 4.1: *AI Recommendation Interface Screenshot*

4.3 System Benchmarks

The APIVUE platform was subjected to load testing using a simulated concurrent user workload to evaluate throughput under stress conditions. At 500 concurrent users, the system maintained an average request throughput of approximately 4,200 requests per second with a 99th-percentile latency below 80 milliseconds for cached responses. No service failures or memory leaks were observed during 30-minute sustained load tests.

The Rust API Gateway demonstrated memory usage approximately 60% lower than an equivalent Node.js-based gateway implementation under the same load, consistent with published benchmarks comparing Rust and Node.js network services. This low memory footprint is particularly significant for high-density deployment scenarios.

4.4 Discussion

The results obtained confirm the core hypothesis of the project: that a Rust-based API Gateway integrated with a semantic AI recommendation engine and a modular NestJS microservices architecture delivers meaningfully better performance and discoverability outcomes than existing platforms.

The cache hit rate during testing was approximately 73% of all requests, indicating that the Redis caching strategy effectively absorbs a significant proportion of traffic before it reaches the database. This not only reduces latency but also reduces load on PostgreSQL, improving the long-term scalability of the platform.

The AI recommendation results demonstrate clear superiority over keyword search, which is particularly significant given that developers frequently search with imprecise or domain-specific natural language rather than exact API names. The ability to understand intent rather than keywords is a foundational advantage of the APIVUE approach.

One observed limitation is that the AI recommendation latency for cache-miss scenarios (150–250 ms) may be perceived as slow by developers expecting

instant results in a search interface. Future work should explore precomputation of embedding indices and more aggressive caching of recommendation results for common query patterns to bring this latency below 50 ms for the majority of cases.

Chapter-5

Conclusion and Future Scope

5.1 Conclusion

This project presented the design and implementation of APIVUE, a next-generation AI-integrated API marketplace engineered to address the fundamental shortcomings of existing API management platforms. The core contributions of this work are as follows:

First, APIVUE demonstrates that it is feasible and practically beneficial to combine a high-performance Rust-based API Gateway with a modular NestJS microservices backend in a unified API marketplace platform. The Rust Gateway provides sub-5-millisecond latency for cached requests and maintains system stability under 500 concurrent users, validating the choice of Rust as the gateway technology.

Second, the integration of semantic search and AI-powered recommendations as first-class platform features — rather than optional add-ons — significantly improves API discoverability. The semantic search engine achieved a Precision@5 of 0.78, substantially outperforming keyword-based alternatives and demonstrating that NLP-driven discovery is a viable and effective approach to API search.

The APIVUE project successfully demonstrates the development of an **AI-integrated API marketplace** designed to improve API discovery, integration, and management for developers. The platform addresses major limitations of traditional API marketplaces such as poor discoverability, lack of intelligent recommendations, and performance bottlenecks in high-traffic environments. By implementing a **Rust-based API Gateway**, the system achieves high performance through efficient request routing, authentication, caching, and rate limiting. The use of **NestJS microservices architecture** ensures modularity, scalability, and fault isolation, making the backend system reliable and maintainable. Additionally, the integration of **PostgreSQL** for structured data storage and **Redis** for caching enhances system efficiency and reduces latency. Furthermore, the incorporation of **AI-powered semantic search and recommendation features** improves the accuracy and relevance of API discovery, enabling developers to find suitable APIs quickly using natural

language queries. Overall, APIVUE provides a modern, scalable, and intelligent solution for API marketplace management, proving the effectiveness of combining AI capabilities with high-performance backend infrastructure.

Third, the Redis caching strategy reduces load on backend services by absorbing approximately 73% of all incoming requests, directly translating to improved scalability and cost efficiency as the platform scales.

Fourth, the modular NestJS microservices architecture (Auth, API, API Key, Analytics) provides clean fault isolation and independent scalability, ensuring that the failure or overload of one service does not cascade to others — a critical property for a production-grade API platform.

In summary, APIVUE successfully demonstrates a viable architecture for the next generation of API marketplaces: one that is intelligent, fast, modular, and developer-centric. The results obtained validate the hypothesis and confirm that the APIVUE approach offers a meaningful advancement over the current state of the art in API management platforms.

5.2 Future Scope

Although APIVUE successfully implements the core features of an AI-integrated API marketplace, there are several improvements and expansions that can be added in the future to make the platform more powerful and industry-ready.

In the future, APIVUE can include a **complete analytics dashboard** that shows real-time API usage, request counts, latency, error rates, and performance reports for both developers and API providers. This will help users monitor their API consumption and improve decision-making.

Another major future enhancement is the development of a **developer playground**, where users can directly test APIs inside the platform without writing code externally. Along with this, APIVUE can provide **automatic SDK generation** in multiple languages like Python, Java, JavaScript, and Go, which will reduce integration time for developers.

APIVUE can also introduce an **API monetization and billing system** where API providers can publish paid APIs with different subscription plans. This

feature will support revenue generation and encourage more providers to publish their APIs on the platform.

Further, the AI system can be enhanced by implementing **AI-powered dynamic rate limiting**, where limits adjust automatically based on traffic patterns and user behaviour. This will increase system security and prevent abuse.

In the future, APIVUE can expand to support **mobile SDK integration** for Android and iOS developers. Additionally, **global CDN integration** can be implemented to provide faster API response times for users from different geographical regions.

7. **Analytics Dashboard:** A real-time observability dashboard providing developers and API providers with live metrics on API usage, error rates, latency distributions, and throughput trends. This will be powered by the existing Analytics microservice with a dedicated time-series data store.
8. **Developer Playground and SDK Auto-Generation:** An interactive sandbox allowing developers to test API calls directly within the APIVUE interface without leaving the platform, paired with automatic generation of client SDKs in multiple programming languages (Python, JavaScript, Go, Java) to accelerate adoption.
9. **Advanced AI-Powered Rate Limiting:** Granular rate limiting strategies that adapt dynamically based on detected usage patterns, enabling differentiated limits for different use cases (batch processing vs. realtime queries) without requiring manual configuration.
10. **API Monetisation Layer:** Built-in billing infrastructure allowing API providers to publish paid APIs with tiered pricing models, and API consumers to manage subscriptions and usage-based billing through a unified payment interface.
11. **Mobile SDK Support:** Native SDKs for iOS and Android platforms, enabling mobile application developers to discover and integrate APIs from APIVUE directly within their mobile development workflow.
12. **Global CDN Integration:** Edge caching of API responses across multiple geographic regions to reduce latency for globally distributed users and improve resilience against regional infrastructure failures.

These planned enhancements collectively represent a roadmap toward making APIVUE the definitive API platform for modern engineering teams —

combining AI intelligence, infrastructure performance, and developer experience into a single, comprehensive solution.

References

1. **R. Fielding**, “Architectural Styles and the Design of Network-based Software Architectures”, Ph.D. Dissertation, University of California, Irvine, 2000.
2. **V. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin**, “Attention Is All You Need”, *NeurIPS*, pp. 5998–6008, 2017.
3. **J. Devlin, M. W. Chang, K. Lee, and K. Toutanova**, “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding”, *NAACL*, pp. 4171–4186, 2019.
4. **V. Karpukhin, B. Oguz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and Y. Yih**, “Dense Passage Retrieval for Open-Domain Question Answering”, *EMNLP*, pp. 6769–6781, 2020.
5. **S. Newman**, “Building Microservices: Designing Fine-Grained Systems”, O’Reilly Media, 2015.
6. **C. Richardson**, “Microservices Patterns: With Examples in Java”, Manning Publications, 2018.
7. **M. Fowler and J. Lewis**, “Microservices: a definition of this new architectural term”, *Martin Fowler Blog*, 2014.
8. **E. Evans**, “Domain-Driven Design: Tackling Complexity in the Heart of Software”, Addison-Wesley, 2003.
9. **D. Merkel**, “Docker: Lightweight Linux Containers for Consistent Development and Deployment”, *Linux Journal*, vol. 2014, no. 239, pp. 1–6, 2014.
10. **B. Burns, B. Grant, D. Oppenheimer, E. Brewer, and J. Wilkes**, “Borg, Omega, and Kubernetes”, *ACM Queue*, vol. 14, no. 1, pp. 70–93, 2016.
11. **M. Jones, J. Bradley, and N. Sakimura**, “JSON Web Token (JWT)”, *RFC 7519*, IETF, pp. 1–30, 2015.
12. **D. Hardt**, “The OAuth 2.0 Authorization Framework”, *RFC 6749*, IETF, pp. 1–76, 2012.
13. **E. Rescorla**, “The Transport Layer Security (TLS) Protocol Version 1.3”, *RFC 8446*, IETF, pp. 1–160, 2018.

14. **S. Klabnik and C. Nichols**, “The Rust Programming Language”, No Starch Press, 2019.
15. **R. Jung, J. Jourdan, R. Krebbers, and D. Dreyer**, “RustBelt: Securing the Foundations of the Rust Programming Language”, ACM, pp. 1–34, 2017.
16. **Actix Web Community**, “Actix Web: A Powerful, Pragmatic, and Extremely Fast Web Framework for Rust”, Actix Documentation, 2024.
17. **NestJS Community**, “NestJS: A Progressive Node.js Framework for Building Efficient Microservices”, NestJS Documentation, 2024.
18. **S. Sanfilippo and P. Noordhuis**, “Redis: An In-Memory Data Structure Store”, Redis Documentation, 2009.
19. **Redis Labs**, “Redis Caching and Performance Optimization Techniques”, Redis Whitepaper, 2022.
20. **PostgreSQL Global Development Group**, “PostgreSQL Documentation: The World’s Most Advanced Open Source Relational Database”, PostgreSQL Docs, 2024.
21. **J. Dean and S. Ghemawat**, “MapReduce: Simplified Data Processing on Large Clusters”, OSDI, pp. 137–150, 2004.
22. **L. Kleinrock**, “Communication Nets: Stochastic Message Flow and Delay”, McGraw-Hill, 1964.
23. **P. Mell and T. Grance**, “The NIST Definition of Cloud Computing”, NIST Special Publication 800-145, pp. 1–7, 2011.
24. **M. Mohri, A. Rostamizadeh, and A. Talwalkar**, “Foundations of Machine Learning”, MIT Press, 2018.
25. **A. Aggarwal and P. Jalote**, “Performance Analysis of API Gateway in Microservices Architecture”, IEEE, pp. 1–6, 2021.
26. **M. Wei, N. S. Harzevili, A. B. Belle et al.**, “A Survey on Query-based API Recommendation,”arXivpreprint, Dec.2023.
27. **Semantics-based API discovery, matching and composition with linked metadata**, ResearchGate, 2020 (formal model for API discovery using semantic).
28. **V. H. Khedkar**, “The Transformative Impact of Artificial Intelligence and Machine Learning on API Management: A Comprehensive Review,” International Journal of Computer Engineering and Technology, vol. 15, no.

- 6, pp. 602–615, 2024.
29. X. Ju *et al.*, “Wis Paper: Your AI Scholar Search Engine,” *arXiv preprint*, Dec. 2025 (research on AI-powered semantic search and recommendation systems).
30. Z. Mousavi, C. Islam, M. A. Babar *et al.*, “Detecting Misuse of Security APIs: A Systematic Review,” *arXiv preprint*, Jun. 2023.